

FOOD ORDERING SYSTEM

WEEK 2 REPORT

Product and Category Management System

Introduction

The Food Ordering System is a database-based system used to manage food items available for ordering. The system organizes food items into different categories such as South Indian, North Indian, Fast Food, Beverages, and Desserts. In Week 2, the main focus is on managing Category and Product information using SQL. The database stores details such as product name, category, price, and stock quantity. SQL constraints are used to maintain the accuracy and consistency of the data.

Objectives

- To create a Category table.
- To create a Product table.
- To establish a relationship between Category and Product.
- To maintain product prices and stock quantities.
- To prevent invalid data using constraints.
- To insert, update, and delete product information.
- To retrieve product and category information using SQL queries.
- To calculate product counts and average prices.
- To identify the highest-priced food item in each category.

Database Tables

Category Table

The Category table stores information about different food categories.

Column	Data Type	Description
CATEGORY_ID	VARCHAR(7)	Unique ID of category
CATEGORY_NAME	VARCHAR(100)	Name of food category
DESCRIPTION	VARCHAR(50)	Description of category

Example Category Data:

CATEGORY_ID	CATEGORY_NAME	DESCRIPTION
C01	South Indian	South Indian food items

C02	North Indian	North Indian food items
C03	Fast Food	Fast food items
C04	Beverages	Hot and cold drinks
C05	Desserts	Sweet food items

```
SQL> CREATE TABLE CATEGORY(CATEGORY_ID VARCHAR(7),CATEGORY_NAME VARCHAR(20),DESCRIPTION VARCHAR(50));
```

Table created.

Product Table

The Product table stores information about food items.

Column	Data Type	Description
PRODUCT_ID	VARCHAR(7)	Unique product ID
PRODUCT_NAME	VARCHAR(150)	Name of food item
CATEGORY_ID	VARCHAR(7)	Category of the product
PRICE	NUMBER(9,2)	Price of food item
STOCK_QUANTITY	NUMBER	Available quantity

Example Product Data:

PRODUCT_ID	PRODUCT_NAME
P01	IDLI
P02	BUTTER NAAN
P03	CHICKEN FRIED RICE
P04	FRESH LIME JUICE
P05	BROWNIE

```
SQL> CREATE TABLE PRODUCT(PRODUCT_ID VARCHAR(7),PRODUCT_NAME VARCHAR(20),CATEGORY_ID VARCHAR(7),PRICE NUMBER(9,2),STOCK_QUANTITY NUMBER);
```

Table created.

Primary Key

A Primary Key uniquely identifies each record in a table. CATEGORY_ID is the primary key of Category, and PRODUCT_ID is the primary key of Product.

```
SQL> ALTER TABLE CATEGORY ADD CONSTRAINT PK_CATEGORY PRIMARY KEY (CATEGORY_ID);
```

Table altered.

```
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

Foreign Key

A Foreign Key establishes a relationship between two tables. CATEGORY_ID in the Product table refers to CATEGORY_ID in the Category table. One category can contain multiple food products.

```
SQL> ALTER TABLE Product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_id);  
Table altered.
```

Constraints Used

NOT NULL Constraint

Category name and product name cannot be empty.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.
```

```
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

CHECK Constraint

The product price must be greater than zero and stock quantity cannot be negative.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK(Stock_Quantity >= 0);  
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_PRICE CHECK (Price > 0);  
Table altered.
```

UNIQUE Constraint

Two categories cannot have the same category name.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);  
Table altered.
```

Data Insertion

Insert Category

This query allows new food categories to be added to the system.

```
SQL> INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description');  
Enter value for category_id: C01  
Enter value for category_name: SOUTH INDIAN  
Enter value for description: Traditional South Indian food  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('C01', 'SOUTH INDIAN', 'Traditional South Indian food')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: C02  
Enter value for category_name: NORTH INDIAN  
Enter value for description: Popular North Indian dishes  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('C02', 'NORTH INDIAN', 'Popular North Indian dishes')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: C03  
Enter value for category_name: FAST FOOD  
Enter value for description: Quick and tasty fast food  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('C03', 'FAST FOOD', 'Quick and tasty fast food')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: C04  
Enter value for category_name: Beverages  
Enter value for description: Hot and cold beverages  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('C04', 'Beverages', 'Hot and cold beverages')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: C05  
Enter value for category_name: DESSERTS  
Enter value for description: Sweet dishes and desserts  
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')  
new 1: INSERT INTO Category VALUES('C05', 'DESSERTS', 'Sweet dishes and desserts')  
  
1 row created.
```

Insert Product

```
INSERT INTO PRODUCT
VALUES ('&PRODUCT_ID',
       '&PRODUCT_NAME',
       '&CATEGORY_ID',
       &PRICE,
       &STOCK_QUANTITY);
```

This query allows new food products to be added to the system.

```
SQL> INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity);
Enter value for product_id: P01
Enter value for product_name: IDLI
Enter value for category_id: C01
Enter value for price: 40
Enter value for stock_quantity: 50
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P01', 'IDLI', 'C01', 40, 50)

1 row created.
```

```

SQL> /
Enter value for product_id: P02
Enter value for product_name: BUTTER NAAN
Enter value for category_id: C02
Enter value for price: 70
Enter value for stock_quantity: 40
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P02','BUTTER NAAN','C02',70,40)

1 row created.

SQL> /
Enter value for product_id: P03
Enter value for product_name: CHICKEN FRIED RICE
Enter value for category_id: C03
Enter value for price: 140
Enter value for stock_quantity: 50
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P03','CHICKEN FRIED RICE','C03',140,50)

1 row created.

SQL> /
Enter value for product_id: P04
Enter value for product_name: FRESH LIME JUICE
Enter value for category_id: C04
Enter value for price: 60
Enter value for stock_quantity: 40
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P04','FRESH LIME JUICE','C04',60,40)

1 row created.

SQL> /
Enter value for product_id: P05
Enter value for product_name: BROWNIE
Enter value for category_id: C05
Enter value for price: 150
Enter value for stock_quantity: 90
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P05','BROWNIE','C05',150,90)

1 row created.

```

Data Retrieval

To display all categories:

```
SELECT * FROM CATEGORY;
```

CATEGOR	CATEGORY_NAME	DESCRIPTION
C01	SOUTH INDIAN	Traditional South Indian food
C02	NORTH INDIAN	Popular North Indian dishes
C03	FAST FOOD	Quick and tasty fast food
C04	Beverages	Hot and cold beverages
C05	DESSERTS	Sweet dishes and desserts

To display all products:

```
SELECT * FROM PRODUCT;
```

```
SQL> SELECT * FROM Product;
```

PRODUCT_ID	PRODUCT_NAME	CATEGORY_ID	PRICE	STOCK_QUANTITY
P01	IDLI	C01	40.00	50
P02	BUTTER NAAN	C02	70.00	40
P03	CHICKEN FRIED RICE	C03	140.00	50
P04	FRESH LIME JUICE	C04	60.00	40
P05	BROWNIE	C05	150.00	90

Insert Operation

Insert values in already viewed table;

```
SQL> INSERT INTO Product (PRODUCT_ID,PRODUCT_NAME, CATEGORY_ID,PRICE,STOCK_QUANTITY) VALUES('P06','CHICKEN BURGER','C03','200','18');

1 row created.
```

Update Operation

The price of a food item can be modified using the UPDATE command.

```
UPDATE PRODUCT
SET PRICE = 200
WHERE PRODUCT_NAME = 'Brownie';
```

This changes the price of Brownie to ₹200.

```
SQL> UPDATE Product SET Price = 200 WHERE Product_Name = 'BROWNIE';

1 row updated.
```

Delete Operation

A product can be removed using the DELETE command.

```
DELETE FROM PRODUCT
WHERE PRODUCT_NAME = 'VEG BURGER';
```

This removes the specified product from the Product table.

```
SQL> DELETE FROM Product WHERE Product_Name = 'FRESH LIME JUICE';

1 row deleted.
```

Join Operation

The following query displays the category name along with product information. It combines information from the Category and Product tables using CATEGORY_ID.

```
SELECT
    C.CATEGORY_NAME,
    P.PRODUCT_NAME,
    P.PRICE,
    P.STOCK_QUANTITY
FROM PRODUCT P
JOIN CATEGORY C
ON P.CATEGORY_ID = C.CATEGORY_ID;
```

```
SQL> SELECT c.Category_Name,p.Product_Name,p.Price,p.Stock_Quantity FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID;
```

CATEGORY_NAME	PRODUCT_NAME	PRICE	STOCK_QUANTITY
SOUTH INDIAN	IDLI	40.00	50
NORTH INDIAN	BUTTER NAAN	70.00	40
FAST FOOD	CHICKEN FRIED RICE	140.00	50
DESSERTS	BROWNIE	200.00	90
FAST FOOD	CHICKEN BURGER	200.00	18

Count Products in Each Category

```
SELECT
    C.CATEGORY_NAME,
    COUNT(P.PRODUCT_ID) AS PRODUCT_COUNT
FROM PRODUCT P
JOIN CATEGORY C
ON P.CATEGORY_ID = C.CATEGORY_ID
GROUP BY C.CATEGORY_NAME;
```

Purpose: This query calculates the number of food products available in each category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	PRODUCT_COUNT
SOUTH INDIAN	1
FAST FOOD	2
NORTH INDIAN	1
DESSERTS	1

Highest-Priced Product in Each Category

```
SELECT
    C.CATEGORY_NAME,
    P.PRODUCT_NAME,
    P.PRICE AS HIGHEST_PRICE
FROM PRODUCT P
JOIN CATEGORY C
ON P.CATEGORY_ID = C.CATEGORY_ID
WHERE P.PRICE = (
    SELECT MAX(P2.PRICE)
    FROM PRODUCT P2
    WHERE P2.CATEGORY_ID = P.CATEGORY_ID
);
```

Purpose: This query finds the most expensive food item in each category. It uses JOIN, MAX(), a subquery, and WHERE.

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_price FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID WHERE p.Price=(SELECT MAX(p2.Price) FROM Product p2 WHERE p2.Category_ID=p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
SOUTH INDIAN	IDLI	40
NORTH INDIAN	BUTTER NAAN	70
DESSERTS	BROWNIE	200
FAST FOOD	CHICKEN BURGER	200

Categories Having More Than 2 Products

```
SELECT
    C.CATEGORY_NAME
FROM PRODUCT P
JOIN CATEGORY C
ON P.CATEGORY_ID = C.CATEGORY_ID
GROUP BY C.CATEGORY_NAME
HAVING COUNT(P.PRODUCT_ID) > 2;
```

Purpose: This query displays categories containing more than two food products.

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID)>2;
```

no rows selected

Average Price of Products

```
SELECT
    C.CATEGORY_NAME,
    AVG(P.PRICE) AS AVG_PRICE
FROM PRODUCT P
JOIN CATEGORY C
ON P.CATEGORY_ID = C.CATEGORY_ID
GROUP BY C.CATEGORY_NAME;
```

Purpose: This query calculates the average price of food items in each category.

```
SQL> SELECT c.Category_Name,AVG(p.Price) AS Avg_Price FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
SOUTH INDIAN	40
FAST FOOD	170
NORTH INDIAN	70
DESSERTS	200

SQL Concepts Used

SQL Concept	Used For
CREATE TABLE	Creating tables
ALTER TABLE	Modifying tables
PRIMARY KEY	Unique identification
FOREIGN KEY	Connecting tables
NOT NULL	Preventing empty values
UNIQUE	Preventing duplicate category names
CHECK	Validating price and stock
INSERT	Adding data
SELECT	Retrieving data
UPDATE	Modifying data
DELETE	Removing data
JOIN	Combining tables
COUNT()	Counting products
MAX()	Finding highest price
AVG()	Finding average price
GROUP BY	Grouping records

HAVING	Filtering groups
Subquery	Finding category-wise maximum

Summary

The Week 2 Food Ordering System successfully implements Product and Category Management using Oracle SQL. The database contains related Category and Product tables with primary keys, a foreign key, and validation constraints. SQL operations such as INSERT, SELECT, UPDATE, DELETE, JOIN, GROUP BY, HAVING, COUNT(), MAX(), and AVG() are used to manage and analyze food product information.